

AutiMate

An AI-assisted mobile application supporting autistic children

The complete project document — what it is, how it works, how it is built, and what it deliberately is not.

Project	AutiMate — Final Year Project, and Alibaba Cloud AI Hackathon Pakistan 2026 entry
Prepared by	Muhammad Hamza Habib — Track D3, AI · Speech · Adaptive Logic
Team	4 people — D1 Frontend & Design · D2 Firebase & Data · D3 AI/Speech/Logic · D4 Integration & Release
Platform	Flutter — Android primary, iOS best-effort
Build window	22–27 August 2026 (hackathon build phase); regional round 28–30 August
Document version	1.0 · 20 August 2026
Status	Reference document for the team, the supervisor, and the judging panel

How to read this document. Sections 1–5 are the product: what AutiMate is and what it does. Sections 6–9 are the engineering: architecture, data, the intelligence, and speech. **Section 10 is a standalone practical guide to checking whether your Android phone has an Urdu voice** — the single riskiest unknown in the project, and the one thing that must be settled before the build starts. Sections 11–21 cover delivery, risk, quality and the demo. The appendices carry the code contracts and the probe app.

Where this comes from. This document consolidates three source documents in the project folder — [AutiMate_Scope_Doc.md](#) (the 16-week product scope), [AutiMate_7Day_Sprint_Plan.md](#) (the four-person build sprint) and [AutiMate_D3_AI_Track_Plan.md](#) (the AI track work order) — into one continuous account, with diagrams. Where the sources disagree, the sprint plan wins for what gets built and the scope document wins for what gets documented.

Contents

Contents	2
1 Executive summary.....	5
1.1 What AutiMate is	5
1.2 The three-sentence version	5
1.3 What makes it different.....	5
2 The problem.....	6
2.1 What the child faces	6
2.2 Why existing tools do not fit.....	6
2.3 What AutiMate is not.....	6
3 Who it is for.....	7
3.1 The roles.....	7
3.2 What a day looks like	7
4 Objectives and how success is measured.....	9
5 The product — eight modules	10
5.1 M1 · Communication Aid (AAC) — the flagship	10
5.2 M2 · Social Communication Training	10
5.3 M3 · Emotion and Expression Recognition.....	11
5.4 M4 · Routine and Flexibility Builder	11
5.5 M5 · Interest-Based Learning	11
5.6 M6 · Sensory-Friendly Environment	11
5.7 M7 · Parent and Teacher Dashboard.....	11
5.8 M8 · Adaptive Support Levels.....	11
5.9 Cross-cutting concerns.....	11
5.10 Explicitly out of scope.....	12
6 How it is built	13
6.1 The layers.....	13
6.2 The folder structure	14
6.3 The one rule that holds it together	14
6.4 Technology stack	14
7 The data.....	16
7.1 Collections.....	16
7.2 Who can read what.....	17

7.3 The grammar fields — a small detail with a large effect.....	17
8 The intelligence — where the AI actually is.....	18
8.1 The honest inventory	18
8.2 The AAC pipeline	19
8.3 Bilingual sentence realisation	19
8.4 Card ranking — recency-weighted frequency.....	20
8.5 The emotion activity engine.....	21
8.6 Adaptive difficulty.....	22
8.7 Expression practice — the on-device machine learning	23
8.8 What was rejected, and why.....	24
9 Speech — the most demo-critical service in the app.....	25
9.1 What the service must do.....	25
9.2 How the Urdu voice is resolved	25
9.3 Queue, barge-in and warm start	26
9.4 The risk this section exists to manage.....	26
10 Checking whether your Android phone has an Urdu voice.....	27
10.1 Why this is the first thing to test.....	27
10.2 Method 1 — the phone's own settings, no code required.....	27
10.3 Method 2 — the probe app (the definitive answer).....	29
10.4 Method 3 — from a laptop over adb (optional)	30
10.5 The three traps	30
10.6 If Urdu is missing — what to try, in order	30
10.7 Insurance worth buying whatever the answer is.....	31
10.8 The checklist to run on the demo phone	31
11 Working without a connection	33
12 Non-functional requirements.....	34
13 Privacy, safety and ethics.....	35
14 The team	36
14.1 The three rules.....	36
15 Delivery plan.....	38
15.1 The six-day build	38
15.2 The 16-week roadmap this is a slice of	39
15.3 If time runs short — the cut order.....	39
16 Risks.....	40

17	Quality — testing and the definition of done.....	42
17.1	What is tested, and how.....	42
17.2	Definition of done, per feature.....	42
18	What you hand over, and in what format.....	43
18.1	Why this question comes up at all.....	43
18.2	What I deliver — the D3 track.....	44
18.3	What the team delivers.....	45
18.4	Two audiences, two packages — one codebase.....	45
18.5	Formats, names, and the things that get lost.....	46
18.6	The submission package.....	46
18.7	If someone asks “where is your model?”.....	46
19	The demonstration.....	48
20	Open questions to settle before the build starts.....	49
21	Glossary.....	50
	Appendix A Interface contracts.....	51
A.1	Speech.....	51
A.2	Sentence assembly.....	51
A.3	Card ranking, emotion activity, adaptive level.....	52
A.4	Expression practice.....	53
	Appendix B The Urdu voice probe.....	54
B.1	The dependency.....	54
B.2	The probe.....	54
	Appendix C Figures and sources.....	55
C.1	Figure index.....	55
C.2	Source documents.....	55

1 Executive summary

1.1 What AutiMate is

AutiMate is a child-friendly Android application that helps autistic children communicate, recognise emotions, follow daily routines, and stay regulated in a sensory environment they can tolerate. It also gives the adults around that child — parents, teachers, therapists — lightweight tools to set activities up and to see what is actually happening over time.

Its flagship feature is an **AAC communication aid**: a grid of picture cards a non-verbal or minimally verbal child taps to build a sentence, which the phone then speaks aloud in **English or Urdu**.

Everything a child touches works **fully offline**, on the device, with no network call in the path between a tap and a spoken word.

1.2 The three-sentence version

A non-verbal child taps two picture cards and the phone says “**I want an apple.**” One toggle later, the same two taps produce grammatical Urdu — میں سیب چاہتا ہوں — with the verb agreeing with the child’s own gender, because the sentence is generated per language rather than translated. It runs in airplane mode, on a mid-range phone, with nothing about the child leaving the device.

1.3 What makes it different

Most comparable tools	AutiMate
English-only, or a machine translation bolted on	Bilingual by construction — English and Urdu are generated separately from the same structure, with full RTL layout
Require an internet connection for speech	Platform text-to-speech on the device; the core child experience never needs a network
Expensive, or subscription-gated	No payments, no subscriptions, no advertising, no third-party analytics on child data
Built for clinicians, with clinical language	Built for the child and the caregiver; makes no diagnostic or therapeutic claim anywhere
Bright, animated, noisy interfaces	A global sensory mode: reduced motion, muted palette, a volume ceiling, no flashing or sudden audio
Opaque “AI-powered” personalisation	Rule-based and explainable — a parent can be told exactly why a level changed or why a card moved

2 The problem

2.1 What the child faces

Autistic children commonly meet a set of daily obstacles that ordinary software makes worse rather than better:

- **Expressing a need verbally** — the want exists, the words do not arrive, and the frustration lands on both child and caregiver.
- **Reading faces and emotional cues** — an ability most software assumes rather than teaches.
- **Unexpected change** — a routine that shifts without warning can end a day.
- **Generic learning material** — content unrelated to the child's actual interests gets no engagement at all.
- **Sensory overload** — bright colour, animation, sudden sound and visual clutter are standard app design and are actively hostile here.

2.2 Why existing tools do not fit

Established AAC and autism-support applications tend to be expensive, English-only, internet-dependent, or designed around a clinician rather than around the child and the family. In Pakistan that combination excludes most families outright: the price is prohibitive, the language is wrong, and the connectivity assumption does not hold. An Urdu-speaking child using an English-only communication aid is being asked to communicate in a language their family does not speak at home.

2.3 What AutoMate is not

AutoMate is a **support and training tool**. It is explicitly **not** a diagnostic, screening, or treatment application, and it makes no clinical claim anywhere in the interface, the documentation, or the presentation. It does not classify severity, it does not label behaviour automatically, and it never tells anyone what a child is feeling. The emotion features teach recognition; they do not report on a real person's emotional state.

3 Who it is for

Figure 1 — Who AntiMate serves, and where it runs

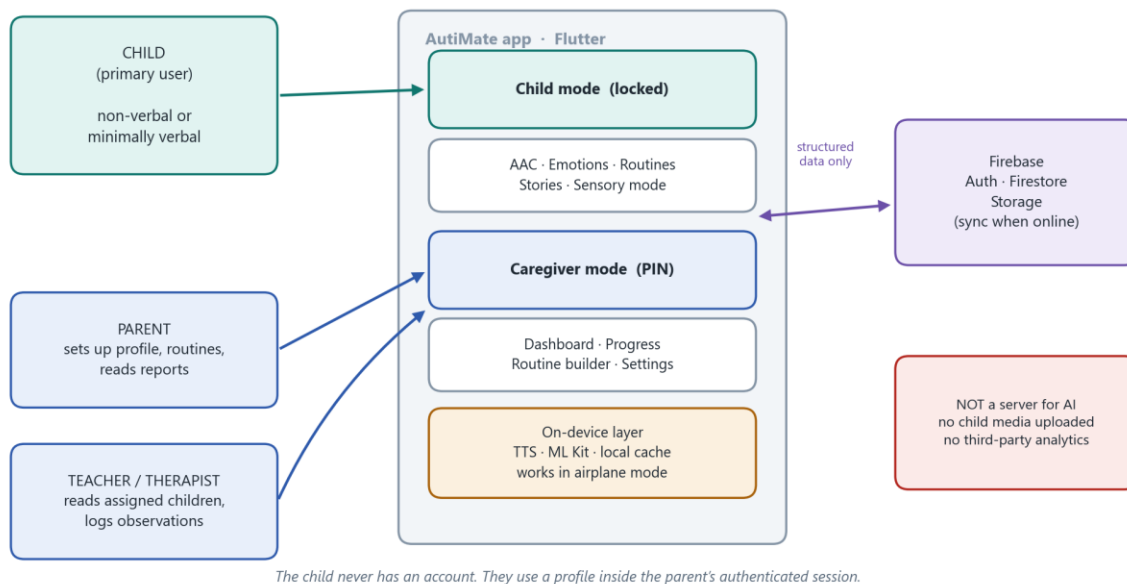


Figure 1 — The three human roles, the two modes of the app, and the strict limit on what ever leaves the device.

3.1 The roles

Role	What they do	What they can reach
Child	Uses the communication aid, emotion activities, routines and calming activities	A simplified, locked child mode. No account, no settings, no way out without a caregiver
Parent	Creates the child profile, configures routines and preferences, adds custom cards, reads reports	Full control of their own children, and nothing belonging to anyone else
Teacher / therapist	Follows progress for assigned children and logs written observations	Read access to assigned children only; write access only to observations
Supervisor / judge	Assesses the project	Not an app role — they receive the demo and this document

A child never holds an account. They use a **profile** inside the parent's authenticated session — which removes an entire class of child-safety and credential problems rather than trying to manage it.

3.2 What a day looks like

Morning. The child opens the routine screen and sees five picture steps: wake, toilet, brush teeth, breakfast, shoes. Each step speaks its own label when tapped. Completing one marks it visibly done.

Five minutes before the routine ends, a transition warning appears — the change is announced, not sprung.

Through the day. The child wants juice. Two taps on the communication grid — [I want] [juice] — and the phone speaks the sentence for them. Cards they use often drift to the front of the grid on their own.

Activity time. A short emotion session: five faces, two to four choices each depending on the child's current level, spoken feedback after every answer, stars at the end.

Evening. The parent opens the dashboard: twelve activities this week, accuracy by emotion, routine completion five days out of seven, and a free-text note they wrote themselves. No automatic behavioural labels, no invented precision.

4 Objectives and how success is measured

Eight objectives, each with an indicator that can actually be checked on a device rather than asserted in a report.

#	Objective	Success indicator
O1	Give non-verbal children a working communication aid	A child can build and speak a request in three taps or fewer
O2	Teach emotion recognition through repeatable activities	Accuracy improves measurably across sessions
O3	Make routines predictable and transitions less stressful	Routine completion is tracked and visualised
O4	Personalise learning around the child's declared interests	A learning path is generated from the interest profile
O5	Reduce sensory overload through a configurable interface	Sensory mode measurably alters motion, sound and contrast
O6	Give caregivers explainable progress insight	A weekly report with no fabricated precision
O7	Work without internet for core child-facing features	Every core module functions fully in airplane mode
O8	Support English and Urdu, including right-to-left layout	Full UI localised; zero hard-coded user-facing strings

5 The product — eight modules

Figure 2 — The eight modules, and what ships in the sprint

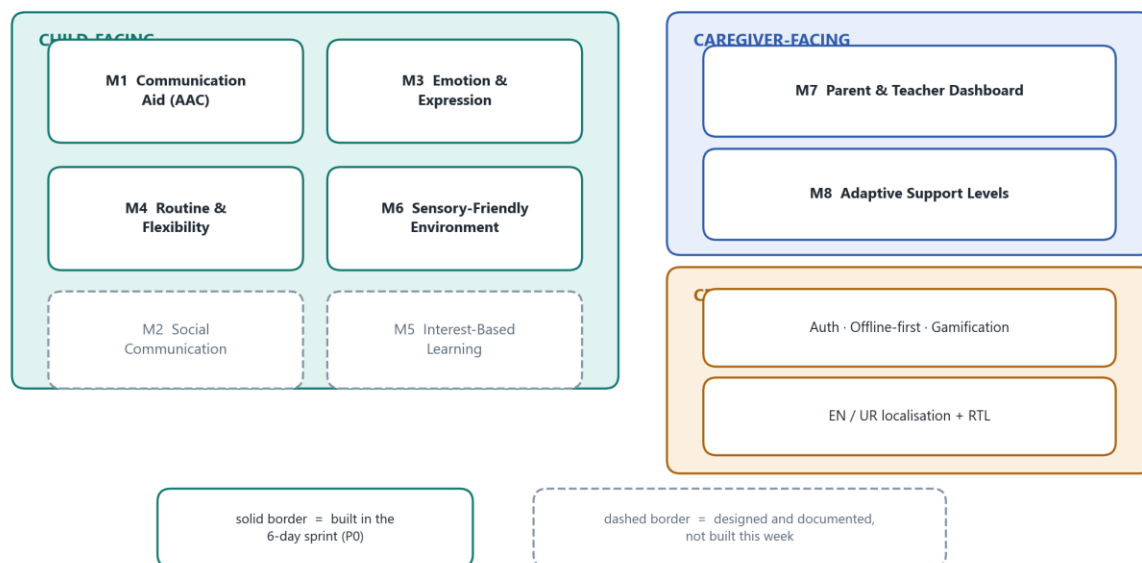


Figure 2 — The eight modules. Solid borders are built in the six-day sprint; dashed borders are designed and documented, not built this week.

5.1 M1 · Communication Aid (AAC) — the flagship

The highest-priority module and the strongest demonstration asset. A grid of large picture cards, organised into categories — food, drinks, emotions, activities, people, places, needs, objects — that a child taps to compose a phrase on a sentence strip. Tapping **Speak** says the whole sentence aloud.

- Every card also speaks its own label the instant it is tapped — immediate auditory feedback is a standard AAC design norm, not a nicety.
- Frequently-used and recently-used cards surface automatically, ranked by the algorithm in section 8.4.
- Parents can add custom cards: an image from the gallery or camera, a label in both languages, and optional recorded audio.
- Tap targets are at least 64 dp, text is minimal, contrast meets WCAG AA.
- The whole module works offline. This is not a stretch goal; it is the module's identity.

5.2 M2 · Social Communication Training

Short illustrated social stories — meeting someone, asking for help, waiting for a turn, going to a shop — each narrated, each followed by a simple comprehension check. Conversation practice is **scripted and branching**, never open-ended generative chat, for the child-safety reasons set out in section 13.

5.3 M3 · Emotion and Expression Recognition

Two halves. **Identification:** the child is shown a face, an emoji or an illustrated scene and picks the emotion from six — happy, sad, angry, surprised, scared, neutral. Difficulty scales through the number of choices, the availability of a hint, and how visually confusable the wrong answers are (section 8.5).

Expression practice: the child is asked to make a face, and an on-device model reads the camera live to fill a progress meter. Camera frames are processed in memory and discarded — never written to storage, never uploaded (section 8.7).

5.4 M4 · Routine and Flexibility Builder

A visual daily schedule built from icon, label, time and optional audio. Steps can be marked complete; local notifications act as reminders; transition warnings announce a change before it happens (“play time ends in five minutes”). **Flexibility training** inserts a small, parent-approved change into a routine the child already knows, with positive reinforcement — practising change in a controlled way rather than only ever avoiding it.

5.5 M5 · Interest-Based Learning

The child’s interests — cars, animals, trains, space — are recorded in their profile and mapped to learning topics through a deterministic, inspectable table. Not a black-box recommender: a caregiver can see exactly why a topic appeared.

5.6 M6 · Sensory-Friendly Environment

A global mode that reduces motion, mutes the palette, lowers brightness, strips clutter and caps volume, plus calming activities: guided breathing, slow visual patterns, gentle sound. Nowhere in the application is there flashing, strobing, or sudden loud audio — including in the reward and feedback moments, which is where such things usually creep in.

5.7 M7 · Parent and Teacher Dashboard

Child profile management, activity history, emotion accuracy over time, routine adherence, and manual observation logging: free text with an optional tag, written by a human. There is **no automatic behavioural labelling** anywhere in the product.

5.8 M8 · Adaptive Support Levels

Three levels — beginner, intermediate, advanced — that adjust choice count, hint visibility, audio assistance, activity complexity and reward frequency. Promotion and demotion follow a simple published rule (section 8.6), and a parent can override and lock the level at any time.

5.9 Cross-cutting concerns

Concern	How it is handled
Authentication	Firebase Authentication for parents and teachers; children use profiles, not accounts

Concern	How it is handled
Persistence	Cloud Firestore for structured data, Firebase Storage for custom card media
Offline	Firestore offline persistence; child-facing features work with no connection at all (section 11)
Gamification	Stars, badges, streaks and progress rings — cooperative and personal, never competitive between children
Localisation	Full English and Urdu, right-to-left verified; no hard-coded user-facing strings anywhere
Security	Firestore security rules enforce data isolation server-side; client checks are treated as UX only

5.10 Explicitly out of scope

Listed so that nobody assumes otherwise halfway through the project, and so the boundary can be defended rather than apologised for.

Excluded	Reason
Clinical diagnosis, screening or severity classification	Outside the project remit; ethically and legally inappropriate
AR / VR experiences	High cost, low marginal value at this scope
Open-ended generative AI chat for children	Safety risk that cannot be adequately guardrailed at this scale
Multi-tenant school or institution management	Enterprise complexity, no project value
Video calling or messaging between users	A significant safety surface for no benefit here
Payments, subscriptions, monetisation	Not applicable
A web dashboard	Optional stretch only; not counted in core deliverables
A bidirectional sync engine with conflict resolution	Last-write-wins is sufficient for this data
Languages beyond English and Urdu	The architecture allows it; the content will not be produced
Wearable or IoT sensor integration	Out of scope
Voice-based emotion recognition	Deferred; only if a later phase finishes early

6 How it is built

6.1 The layers

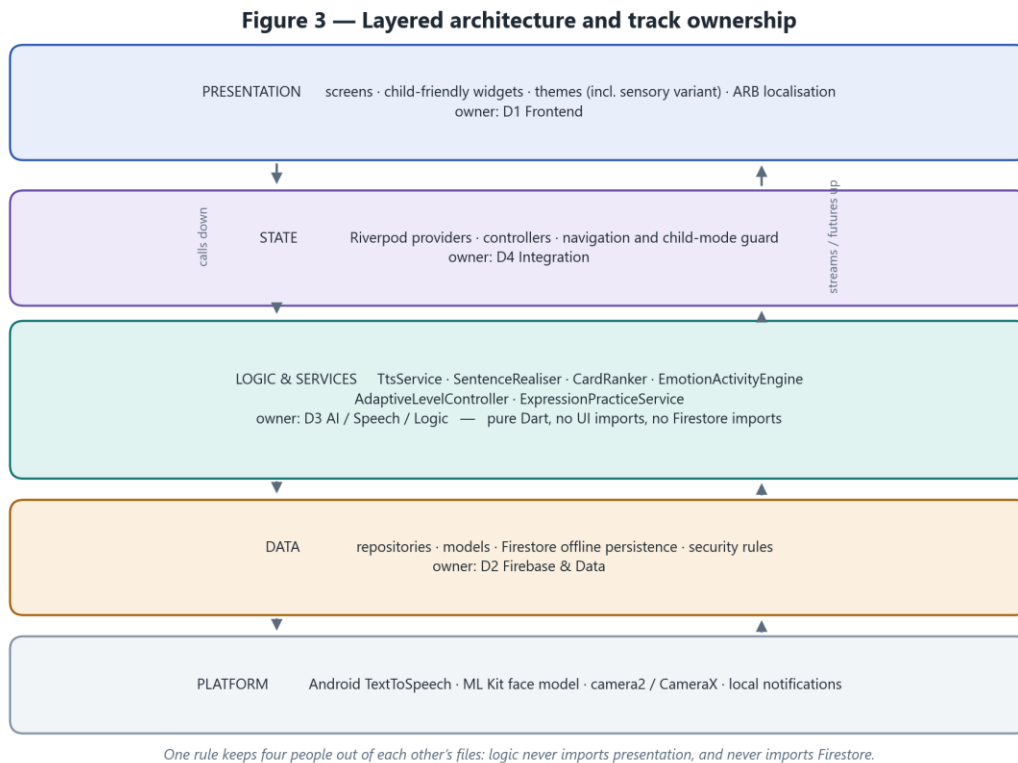


Figure 3 — Five layers, each owned by one person. The ownership boundary is what stops four people editing the same file in a six-day sprint.

The application is a feature-based Flutter project with a conventional downward dependency: presentation calls state, state calls logic and data, and results travel back up as futures and streams. The unusual part is not the shape — it is that each layer has a single named owner for the duration of the sprint.

6.2 The folder structure

Figure 4 — Feature-based project structure



Figure 4 — Feature-based structure. Shared machinery lives in `core/`, each feature owns its own vertical slice.

6.3 The one rule that holds it together

Logic and services live in `lib/core/services/` and `lib/features/*/logic/`, and depend only on Dart, their own packages, and the repository **interfaces**. They import nothing from `presentation/`, and they never touch Firestore directly. If a piece of logic needs a widget, the design is wrong. That single rule is what lets the AI track be written, tested and refactored while three other people are calling into it.

6.4 Technology stack

Layer	Technology	Why this one
App framework	Flutter (Dart)	One codebase, Android and iOS, fast UI iteration
State management	Riverpod	Compile-safe, testable, and the team standard
Authentication	Firebase Authentication	Managed and secure; no hand-rolled auth code
Database	Cloud Firestore	Offline persistence is built in, which serves the offline-first goal directly
Media storage	Firebase Storage	Custom card images and recorded audio
Notifications	Local notifications	Routine reminders are device-local; no server needed

Layer	Technology	Why this one
Speech output	Platform TTS via flutter_tts	Native, offline-capable, free — see section 9
On-device ML	Google ML Kit face detection	Ships a working model, ~30–60 ms per frame, no network
Charts	fl_chart	One dashboard chart; no heavier dependency justified
Design	Figma	UI and UX planned before implementation

Dependency rule. Every package added to the project requires a one-line written justification in the pull request. If Flutter, Firebase, or an existing dependency already does the job, no new package is added. [build_runner](#) and code generation are excluded outright — in a six-day sprint a single generation failure can cost half a day, and the model classes are small enough to write by hand.

7 The data

Figure 5 — Firestore data model (a collection is created only when its feature lands)

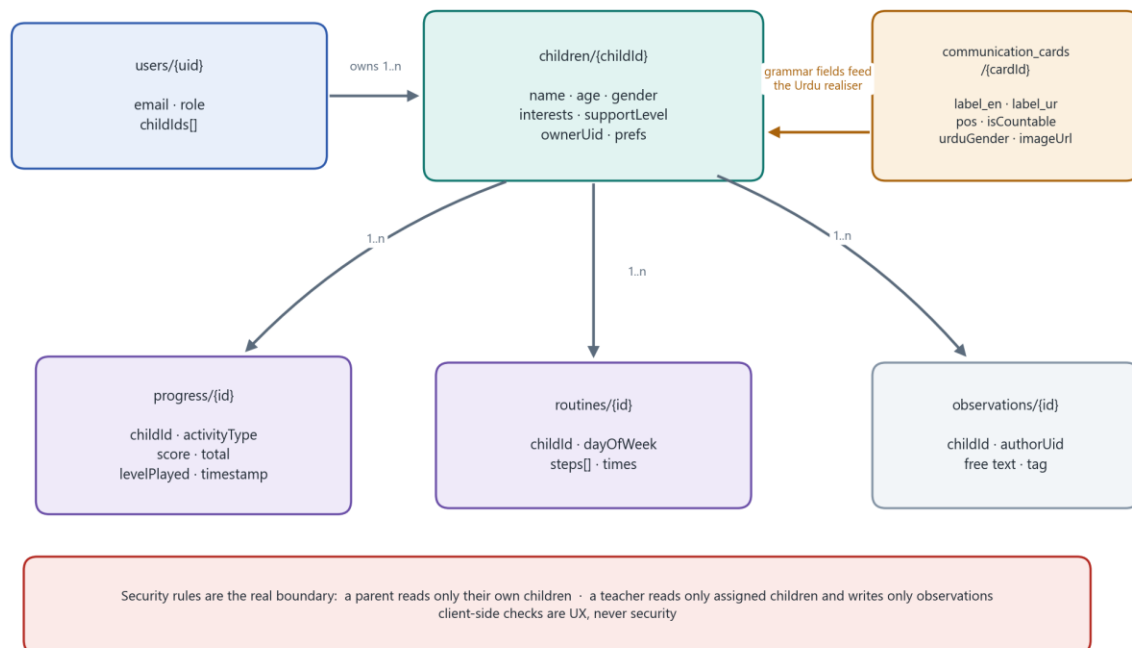


Figure 5 — The Firestore collections and how they relate. A collection is created only when the feature that needs it is implemented.

7.1 Collections

Collection	Holds	Created in
users/{uid}	Parent and teacher accounts, role, linked children	Phase 1
children/{childId}	Child profile: name, age, gender, interests, preferences, support level	Phase 1
communication_cards/{cardId}	Default and custom AAC cards, including the grammar fields	Phase 2
social_stories/{id}	Story content and steps	Phase 2
activities/{id}	Activity definitions for emotion and learning modules	Phase 2
progress/{id}	Per-child activity results and scores	Phase 2
routines/{id}	Routine templates and daily steps	Phase 3
learning_content/{id}	Interest-mapped learning items	Phase 3
observations/{id}	Manual caregiver observation logs	Phase 4

7.2 Who can read what

- A parent reads and writes only their own children's data.
- A teacher reads only children explicitly assigned to them, and writes only observations.
- No user can read another user's records under any circumstance.
- **Client-side checks are user experience. The Firestore security rules are the actual boundary.**

7.3 The grammar fields — a small detail with a large effect

Each seeded communication card carries four extra fields that most card models would not have: `pos` (part of speech), `isCountable`, `startsWithVowelSound`, and `urduGender`. They cost one line each in the seed script and they are what allow the sentence engine in section 8.3 to produce **"I want an apple"** rather than "I want apple", and میں کتاب چاہتی ہوں rather than a word-for-word transliteration. Without them the bilingual engine degrades to naive word-joining, and the strongest ten seconds of the demo disappears.

8 The intelligence — where the AI actually is

This section exists because the question “**where is the AI?**” will be asked, by a supervisor and by a judge, and it deserves a precise answer rather than a marketing one.

8.1 The honest inventory

Capability	What actually ships	Honest label
On-device machine learning	ML Kit face detection driving live expression practice	Real ML , on-device, no network
Bilingual sentence generation	Two language-specific surface realisers over a shared structure	Rule-based NLG , not ML — and correct to say so
Personalisation	Recency-weighted frequency ranking of communication cards	Algorithmic , explainable in one line of maths
Adaptive difficulty	A pure-function state machine over recent outcomes	Rule-based , chosen deliberately over a learned policy
Speech synthesis	Platform TTS, bilingual, offline, queued	Platform capability , engineered around carefully
Deliberately not built	Custom emotion classifier · speech-to-text · generative chat	Designed and documented as future work

The rule for the room: never claim a model you cannot demonstrate. A team that says “*we used machine learning only where it earned its place, and here it is running live in airplane mode*” is more credible than one claiming a classifier it cannot show. Restraint is defensible; overreach is not.

8.2 The AAC pipeline

Figure 6 — The AAC pipeline: from a tap to a spoken sentence

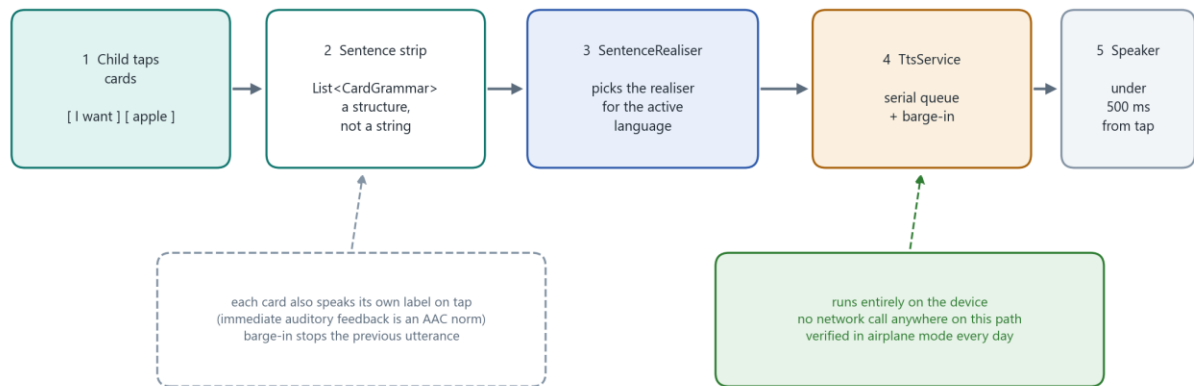


Figure 6 — Tap to speech, with nothing on the path that needs a network.

The important design decision is at step 2. The sentence strip is **not a string**. It is a list of card structures, which is what makes step 3 possible: switching language mid-session re-renders the same taps through a different grammar instead of translating text that has already lost its structure.

8.3 Bilingual sentence realisation

Figure 7 — One structure, two grammars: bilingual sentence realisation

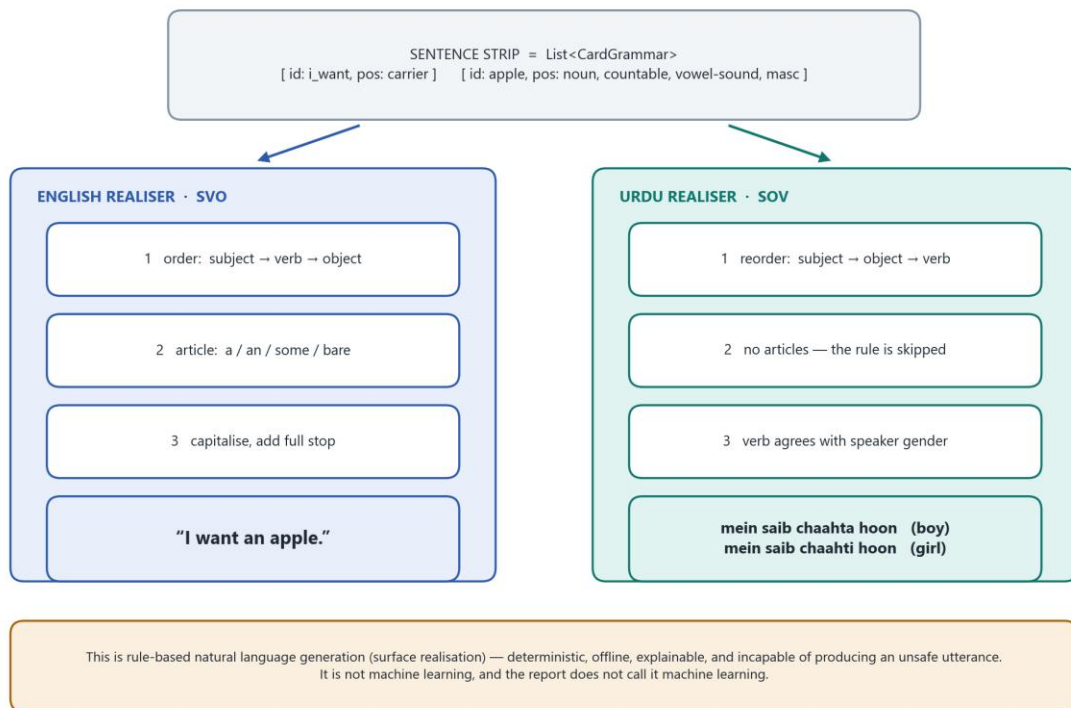


Figure 7 — One structure, two grammars. English is SVO with article selection; Urdu is SOV with speaker-gender verb agreement.

English is subject–verb–object: *I want an apple*. Urdu is subject–object–verb: میں سیب چاہتا ہوں — literally *I apple want-am*. A word-by-word translation of the English strip therefore produces broken Urdu. Two independent realisers, each consuming the same structure, produce grammatical output in both.

The Urdu verb also agrees with the **speaker's** gender, taken from the child's profile — so a boy's app says چاہتا and a girl's says چاہتی. That is a two-hour implementation detail that reads as real linguistic engineering, because it is.

Worked examples — the test table written on day two

Sentence strip	English output	Urdu output	Rule exercised
[I want] [apple]	I want an apple.	میں سیب چاہتا ہوں	"an" before a vowel sound; SOV reorder
[I want] [water]	I want some water.	میں پانی چاہتا ہوں	Uncountable noun takes "some"; Urdu has no article
[I want] [book]	I want a book.	میں کتاب چاہتی ہوں	Consonant article; feminine speaker agreement
[I feel] [happy]	I feel happy.	میں خوش ہوں	Adjective predicate, no object
[apple]	Apple.	سیب	Single bare noun — capitalise, no article
[] (empty strip)	— no speech, no crash —	— no speech, no crash —	The case that breaks live demos

Honest framing. This is rule-based natural language generation — surface realisation. It is not machine learning and the report does not call it machine learning. It is the correct tool: deterministic, offline, explainable, and structurally incapable of producing an unsafe utterance, which matters when the user is a child.

8.4 Card ranking — recency-weighted frequency

Sorting the "frequently used" row by raw usage count fails in an obvious way: a card used forty times last month outranks the card the child needs today. The fix is exponential recency decay.

$$\text{score}(\text{card}) = \sum \text{ over every use } \exp(-\lambda \cdot \text{ageInDays})$$

$$\lambda = \ln(2) / 7 \quad \# \text{ a use is worth half as much after seven days}$$

Sum the decayed weight of every use of a card, sort descending, take the top eight. It needs no training data, runs offline in microseconds, self-corrects when a child's needs change, and — the property that actually matters — a caregiver can be told exactly why any given card is in the top row.

8.5 The emotion activity engine

Figure 8 — Emotion activity: how a question is built and scored

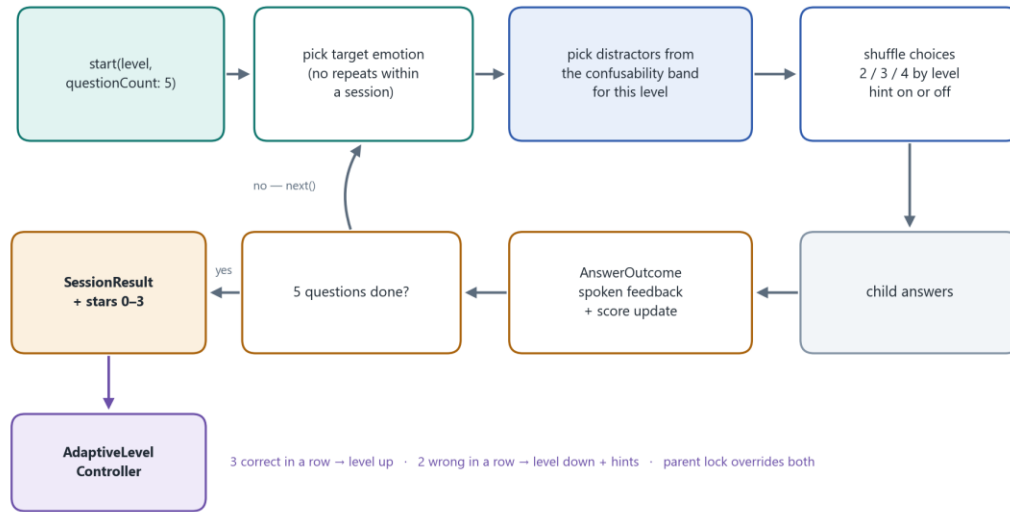


Figure 8 — A five-question session: how each question is assembled, scored, and turned into a persisted result.

Random wrong answers make a poor teaching activity. *Happy* against *sad* is trivial at every level, so the child stops learning. Instead the engine draws distractors from a hand-authored **confusability matrix** — how easily each pair of emotions is mistaken for the other, grounded in the visual features that separate them: mouth curvature, eyebrow angle, eye aperture.

	happy	sad	angry	surprised	scared	neutral
happy	—	0.1	0.1	0.3	0.1	0.4
sad	0.1	—	0.5	0.1	0.6	0.4
angry	0.1	0.5	—	0.2	0.5	0.3
surprised	0.3	0.1	0.2	—	0.7	0.2
scared	0.1	0.6	0.5	0.7	—	0.2
neutral	0.4	0.4	0.3	0.2	0.2	—

Level	Choices	Distractors drawn from	Goal
Beginner	2	The lowest confusability band — happy against sad	Success. The child needs to win
Intermediate	3	The middle band	Consolidation
Advanced	4	The highest band — scared against surprised, sad against scared	Genuine discrimination

When a judge asks how difficulty increases, “more options appear” is a weak answer. “The wrong answers become visually harder to distinguish, along a matrix I can show you” is a strong one.

8.6 Adaptive difficulty

Figure 9 — Adaptive difficulty: a pure function, not a learned policy

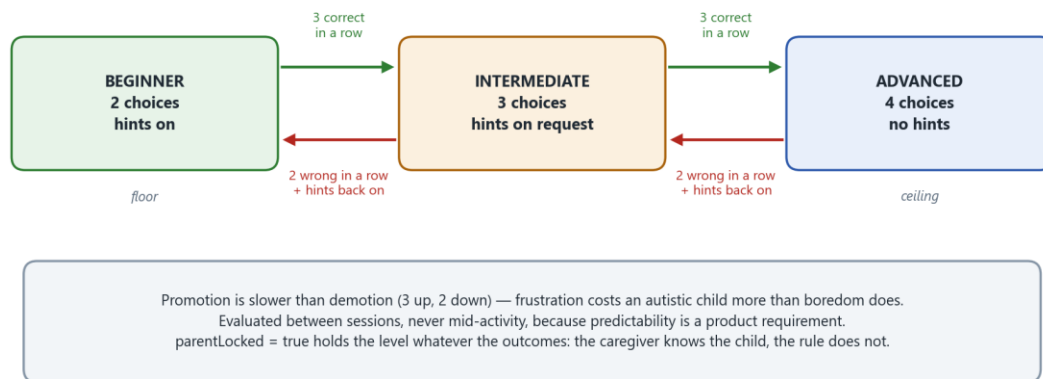


Figure 9 — Three states, four transitions, one override. Every possible transition fits on a whiteboard.

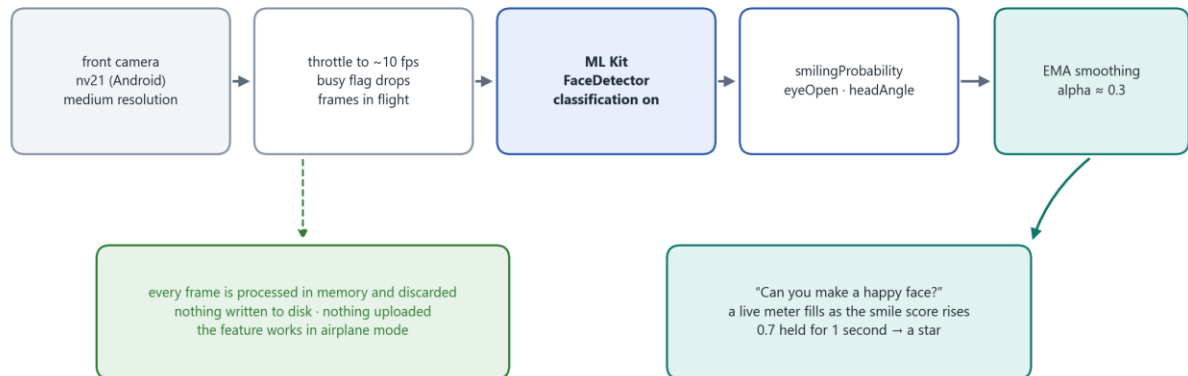
Four design choices here are deliberate, and each has an answer ready if questioned:

- **Promotion is slower than demotion** — three correct to go up, two wrong to come down. Frustration costs an autistic child more than boredom does, so the system errs toward success.
- **Evaluation happens between sessions, never mid-activity.** Difficulty shifting under a child during an activity is disorienting, and predictability is a core product requirement.
- **The parent lock is absolute.** The caregiver knows the child; the rule does not.
- **It is a pure function** — state in, state out, no hidden fields — so every transition is unit tested and enumerable.

“We chose a rule-based adaptive system deliberately. A parent can understand exactly why the level changed. A learned policy would need training data we do not ethically have, and would be unexplainable to the person who most needs to trust it.” That answer is stronger than any model that could be trained in a week.

8.7 Expression practice — the on-device machine learning

Figure 10 — Expression practice (P1): on-device face detection



ML Kit was chosen over a custom TFLite classifier: about four hours of work instead of three to four uncertain days, and it demos better live.

Figure 10 — The camera pipeline. Frames live for one detection pass and are then discarded.

The activity is simple: **“Can you make a happy face?”** A meter fills as the smile probability rises; holding above 0.7 for one continuous second awards a star. It is live, responsive, on-device, and visible from across a room — which is exactly why it must not be started before everything else is finished.

Why ML Kit rather than a custom TFLite classifier

	Custom TFLite model	ML Kit face detection
Setup time	3–4 days, uncertain	About four hours
Runs on device	Yes	Yes
Privacy	Frames stay local	Frames stay local
Latency	Variable	~30–60 ms per frame
Output	Six-class emotion, if it works	Smile probability, eye-open probability, head angle
Risk	High — may end the week with nothing	Low — ships

ML Kit gives `smilingProbability` and `leftEyeOpenProbability` directly. That is enough for a genuinely strong activity with a live meter — which demonstrates **better** than a six-class classifier, because the evaluator can watch it respond in real time. It is still real on-device machine learning, still privacy-preserving, still zero network calls. The honest report wording is: *“expression practice uses on-device ML Kit face detection to derive expression signals locally; a fine-tuned multi-class emotion classifier is documented as future work.”*

Implementation notes that save hours

- Construct the camera controller with `ImageFormatGroup.nv21` on Android and `bgra8888` on iOS. Guessing the format is the single biggest time sink in this integration.
- `FaceDetectorOptions(enableClassification: true)` is required — without it `smilingProbability` is null. Leave contours and landmarks off; they cost latency and are not used.
- Throttle with a **busy flag**, not a timer: drop incoming frames while a detection is in flight, or the queue backs up and latency climbs until the app stalls.
- Smooth with an exponential moving average ($\alpha \approx 0.3$) before the value reaches the UI. Raw per-frame probability jitters and the meter looks broken even when it is working.
- Dispose the detector and stop the stream in `dispose()`. A live camera stream behind a navigation push is a battery drain and an obvious bug in questioning.

8.8 What was rejected, and why

Rejected	Reason
A custom-trained TFLite emotion classifier	Three to four days of uncertain work with a real chance of ending the week with nothing. Documented as future work
Cloud speech synthesis of any provider	Breaks the offline requirement, adds latency and cost, and sends a child's utterances to a server
Open-ended LLM chat for the child	Out of scope on safety grounds; it cannot be adequately guardrailed at this scale
A learned adaptive-difficulty policy	No ethically obtainable training data, and unexplainable to the parent who must trust it
Speech-to-text	Cut from the sprint scope; designed and documented, not built
A translation API for Urdu	Would produce word-order errors exactly where they matter most. Per-language realisation is both cheaper and better

9 Speech — the most demo-critical service in the app

If the text-to-speech service fails, the flagship feature fails and the demonstration is over. It is therefore built first, on day one, and verified on physical hardware every single day thereafter.

9.1 What the service must do

Requirement	Why
Speak English and Urdu	The product claim; Urdu is the accessibility argument
Start speaking under 500 ms after a tap	Slower than that and a child loses the connection between action and sound
Work offline	The core promise; venue wifi cannot be trusted at a demo either
Never throw	A crash mid-utterance ends a demonstration. Failures surface as a state, not an exception
Queue and barge-in	Five fast taps must produce five clean utterances, not garbled overlap
Respect sensory mode	A volume ceiling and a slower rate, driven from settings — not from the theme

9.2 How the Urdu voice is resolved

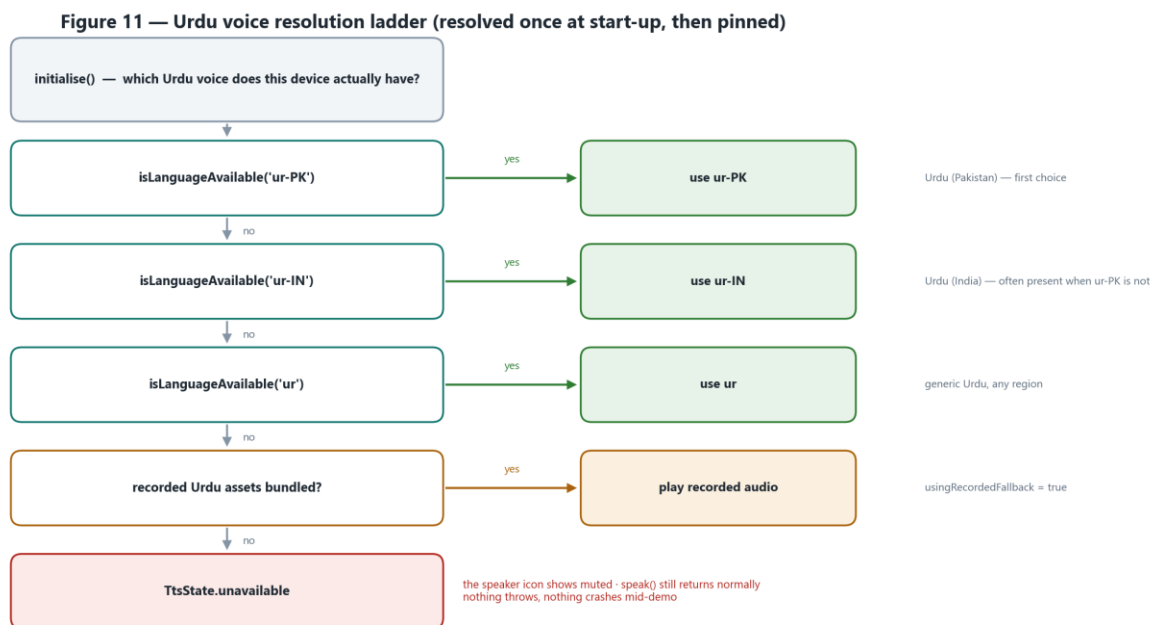


Figure 11 — The fallback ladder. It runs once at start-up; every later call uses the pinned result.

Locale resolution runs once inside `initialise()` and the winning locale is cached — probing on every utterance would add latency to the one path that cannot afford it. The caller never learns which rung

answered: whether the audio came from a platform voice or a recorded asset is entirely the service's business.

9.3 Queue, barge-in and warm start

- **Serial queue.** A single loop drains a queue of utterances, with `awaitSpeakCompletion(true)` so that speaking genuinely awaits — which is what makes ordering possible at all.
- **speak() = clear the queue, stop, then enqueue** — barge-in. **enqueue() = append**, used for routine step sequences.
- **Warm start.** The first utterance on Android pays an engine-initialisation cost that can exceed the 500 ms budget on its own. During `initialise()` the service sets the language and speaks a single space at zero volume. First-tap latency is then measured with a stopwatch and written down — it is a real non-functional requirement that can be quoted.
- **Child-appropriate prosody.** Speech rate around 0.45–0.5 on Android — the platform default is far too fast for a child — pitch 1.0, and a volume ceiling that sensory mode can lower further.

9.4 The risk this section exists to manage

Urdu text-to-speech availability varies by device, by Android version, by region and by which speech engine happens to be installed. It is the single highest-likelihood, highest-impact risk in the whole project, and it is retired before the build starts rather than discovered during it. Section 10 is the procedure for retiring it.

10 Checking whether your Android phone has an Urdu voice

This section is written to be used on its own, phone in hand. It is the procedure for **Day 0** — before any application code is written — and it should be repeated on every device that will be used for the demonstration.

10.1 Why this is the first thing to test

Urdu speech is the accessibility claim the whole product rests on. Unlike English, an Urdu voice is **not guaranteed** to exist on an Android phone: availability depends on the manufacturer, the Android version, the region, which speech engine is installed, and whether the voice data has ever been downloaded. Two phones sitting on the same desk can give different answers.

The cost of discovering this on Day 0 is a couple of hours and a documented decision. The cost of discovering it on Day 5 is the demonstration.

Figure 12 — How to find out whether your Android phone has an Urdu voice

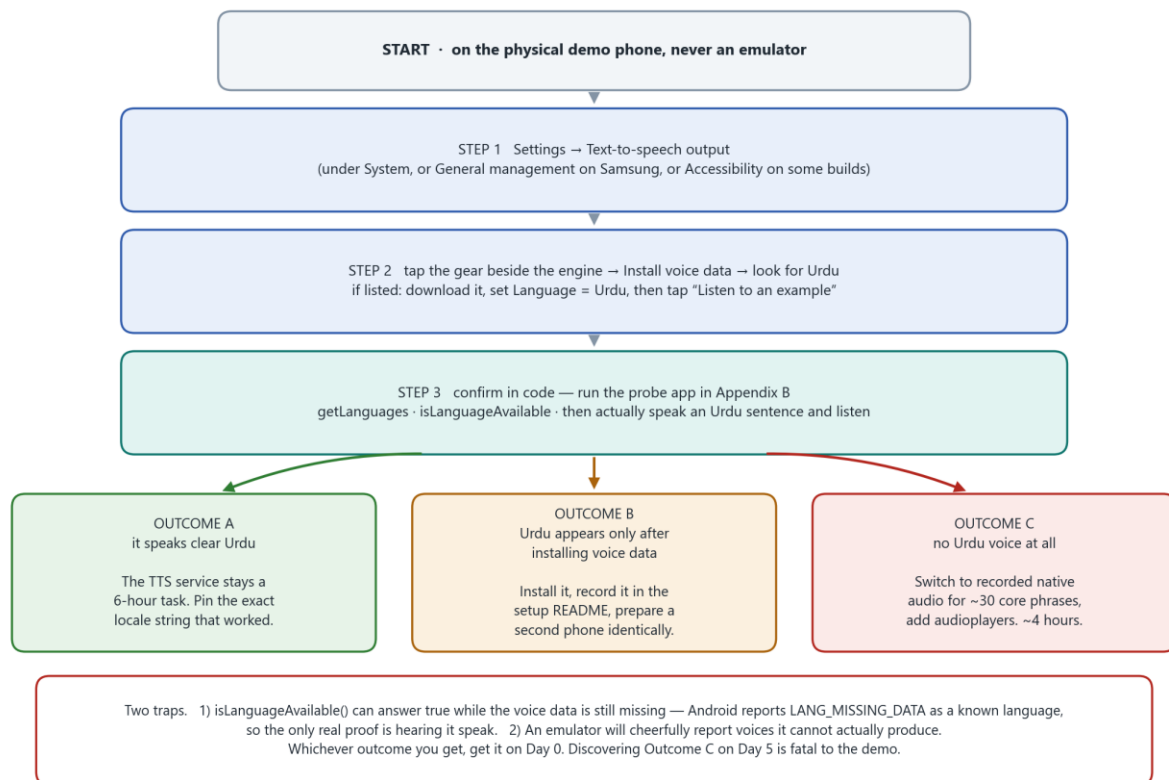


Figure 12 — The whole procedure on one page: the settings route, the probe that confirms it, three possible outcomes, and the traps that produce a false positive.

10.2 Method 1 — the phone's own settings, no code required

Start here. It takes about five minutes and it is also how you **fix** the problem if the voice is simply not downloaded yet.

Step 1 — find the text-to-speech screen

The path differs by manufacturer. Find whichever of these matches your phone:

Phone	Settings path
Stock Android / Pixel / Motorola	Settings → Accessibility → Text-to-speech output (older builds: Settings → System → Languages & input → Text-to-speech output)
Samsung (One UI)	Settings → General management → Text-to-speech
Xiaomi / Redmi / POCO (MIUI, HyperOS)	Settings → Additional settings → Accessibility → Text-to-speech output
Oppo / Realme / vivo / OnePlus	Settings → Additional settings → Accessibility → Text-to-speech output
Infinix / Tecno / itel (HiOS, XOS)	Settings → System → Languages & input → Text-to-speech output
Any phone, if lost	Open Settings, use the search box, and type text-to-speech

Step 2 — note which engine is selected

At the top of that screen is **Preferred engine**. It will usually be one of:

- **Speech Recognition & Synthesis** (package [com.google.android.tts](https://play.google.com/store/apps/details?id=com.google.android.tts)) — Google’s engine, the one to prefer.
- **Samsung text-to-speech engine** ([com.samsung.SMT](https://play.google.com/store/apps/details?id=com.samsung.SMT)) — common on Samsung devices; its language list is narrower.
- An OEM engine, or nothing at all on very stripped-down builds.

Write down which engine is selected. The answer to “does this phone have Urdu?” is really “does **this engine on this phone** have Urdu?” — switching engines can change the answer entirely.

Step 3 — look for Urdu in the voice data

1. Tap the **gear / settings icon** next to the preferred engine.
2. Tap **Install voice data** (on some builds this is under a **Voice data** or **Languages** entry).
3. Scroll the language list and look for **Urdu**. It may appear as *Urdu*, اردو, *Urdu (Pakistan)* or *Urdu (India)*.
4. If it is there, tap it and **download** it. A voice is often a 20–60 MB download; some engines offer several quality variants — take the one marked for offline use if the choice is offered.
5. If it is **not** in the list at all, do not conclude anything yet — go to section 10.6 first, then re-check.

Step 4 — make it actually speak

1. Go back to the main text-to-speech screen.
2. Set **Language** to **Urdu** (turn off “use system language” if that switch is present).
3. Tap **Listen to an example / Play**.
4. **Listen**. Do you hear Urdu, in an intelligible voice? Not English, not silence, not a burst of noise?

A sample that plays in English while the language is set to Urdu means the engine fell back — that is a **negative** result, not a positive one.

Step 5 — repeat it in airplane mode

Turn **airplane mode on** and press **Listen to an example** again. Some Google voices are network-backed: they sound excellent while online and produce nothing at all offline. AutoMate's core claim is that the child-facing features work with no connection, so **a voice that only speaks online does not count as a pass**. This single step catches the most expensive false positive in the whole procedure.

10.3 Method 2 — the probe app (the definitive answer)

The settings screen tells you what the phone believes. The probe tells you what the **app** will actually get when it calls the platform, which is the thing that matters. A complete, runnable probe is in the project at [tools/urdu_tts_probe/](#) — see Appendix B for the core of it.

```
cd tools/urdu_tts_probe
flutter create .          # generate android/ for this pubspec, once
flutter pub get
flutter run               # with a PHYSICAL Android device connected
```

The probe prints, on the phone's own screen so you can read it without a laptop:

- every text-to-speech **engine** installed, and which is the default;
- every **locale** the current engine reports, with any beginning **ur** highlighted;
- direct `isLanguageAvailable` answers for **ur-PK**, **ur-IN** and **ur**, in the order the real service will try them;
- every named **voice** that looks Urdu — including ones whose data has not been downloaded;
- and it gives you buttons to **speak an Urdu sentence** and hear the result.

The Urdu test sentence is **السلام علیکم، میں سیب چاہتا ہوں** — a greeting plus the exact phrase the AAC demonstration will produce, so the probe tests the real thing rather than an abstraction.

What a good result looks like

```
ENGINES INSTALLED
com.google.android.tts
default engine: com.google.android.tts

LOCALES REPORTED: 74
URDU LOCALE FOUND: ur-PK

DIRECT AVAILABILITY CHECKS
isLanguageAvailable("ur-PK") = true
isLanguageAvailable("ur-IN") = false
isLanguageAvailable("ur")    = true

VOICES: 312 total, 2 look Urdu
{name: ur-pk-x-urm-local, locale: ur-PK}
```

The suffix **-local** on a voice name is a good sign: it indicates a voice that synthesises on the device rather than over the network. A voice ending **-network** will not work in airplane mode.

What a bad result looks like

```
LOCALES REPORTED: 41
NO locale starting with "ur"  <-- this is the bad outcome

DIRECT AVAILABILITY CHECKS
isLanguageAvailable("ur-PK") = false
isLanguageAvailable("ur-IN") = false
isLanguageAvailable("ur")    = false
```

10.4 Method 3 — from a laptop over adb (optional)

Useful for recording exactly what was tested, and for checking several phones quickly.

```
adb devices                                # confirm the phone is attached
adb shell settings get secure tts_default_synth # which engine is the default
adb shell pm list packages | grep -i tts      # which engines are installed at all
adb shell dumpsys package com.google.android.tts | grep -i versionName
```

Be honest about what adb can and cannot tell you. These commands identify the engine and its version. They do **not** reliably reveal which voice data has been downloaded — that lives in the engine's private storage. adb is for recording the environment; the probe and your own ears are for the verdict.

10.5 The three traps

Trap	What happens	Defence
A query says yes, the phone says nothing	Android reports language support at several levels — available, country-available, <i>data missing</i> , not supported — and different wrappers collapse those differently. A voice can also be listed with its data not yet installed.	Treat every query as encouraging, not conclusive . Hearing the sentence is the only proof.
The emulator lies	Emulator images happily report voices they cannot produce, and audio routing adds its own failures.	Test on the physical demo device , every time. This is why Day 0 exists.
The voice is network-backed	A high-quality Google voice may synthesise server-side. It sounds perfect in the room with wifi and silent in airplane mode.	Always run the test twice — once online, once in airplane mode. Only the offline pass counts.

10.6 If Urdu is missing — what to try, in order

#	Try this	Effort	Notes
1	Install or update Speech Recognition & Synthesis from the Play Store, then re-check	5 min	Some OEM builds ship an old Google engine with a much shorter language list. This alone fixes many phones
2	Gear → Install voice data → Urdu → download, then re-run the probe	10 min	The most common cause of a false negative is simply that nobody ever downloaded it

#	Try this	Effort	Notes
3	Switch the preferred engine and re-check	5 min	The Samsung engine and the Google engine have different language sets on the same phone
4	Install an alternative engine — eSpeak NG or RHVoice	15 min	eSpeak NG is open source, fully offline and does support Urdu, but it is robotic. Acceptable for development; a poor choice for a child-facing demo
5	Try a different phone	varies	Borrow two or three. Record brand, model, Android version and engine version for whichever works
6	Record native Urdu audio for ~30 core phrases	~4 h	The documented fallback. Add audioplayers , bundle the assets, and route the recorded path behind the same service interface

Option 6 is not a defeat. For a child-facing accessibility application, a recorded native speaker is arguably *better* than a synthetic voice: clearer, warmer, and consistent every time. The honest framing is “Urdu speech uses recorded native-speaker audio for the core vocabulary, with platform text-to-speech wherever a device voice is available.” That is a design decision, not an apology — and because the fallback sits behind the same [TtsService](#) interface, no calling code changes.

10.7 Insurance worth buying whatever the answer is

- **Record the top ten phrases anyway.** Even with a working device voice, an overnight system update can change what the phone has. An hour of recording is cheap insurance against that.
- **Set up a second phone identically** and verify it the same way. One phone is a single point of failure for an in-person demonstration.
- **Pin the exact locale string** that worked — [ur-PK](#) and [ur-IN](#) are not interchangeable — and put it in the setup README with the engine version.
- **Turn off automatic system updates** on the demo device once it is verified.
- **Re-verify on the morning of the demo.** Sixty seconds, and it has caught worse.

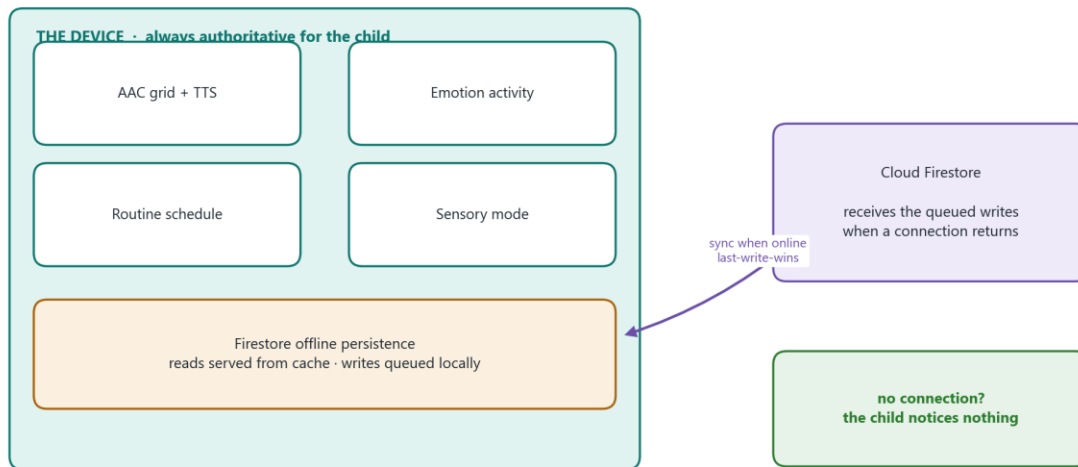
10.8 The checklist to run on the demo phone

✓	Check	Record
☐	Physical device, not an emulator	brand / model / Android version
☐	Text-to-speech settings screen located	path used
☐	Preferred engine identified	package + version
☐	Urdu appears under Install voice data	yes / no
☐	Urdu voice data downloaded	size / variant
☐	“Listen to an example” speaks Urdu — online	yes / no
☐	“Listen to an example” speaks Urdu — airplane mode	yes / no
☐	Probe app run; locale list captured	screenshot

✓	Check	Record
☐	ur-PK / ur-IN / ur availability recorded	true / false each
☐	Probe spoke the Urdu sentence aloud — airplane mode	yes / no
☐	Outcome decided: A, B or C	A / B / C
☐	Result posted to the team the same day	date / time
☐	Second device verified identically	brand / model

11 Working without a connection

Figure 13 — Offline-first: what happens with no internet



Airplane mode is part of the daily test routine, not an edge case discovered on demo day.

Figure 13 — With no network, the child notices nothing. Writes queue locally and reconcile later.

Offline is not a degraded mode bolted on at the end; it is the default the child-facing features are designed around. Firestore's offline persistence serves reads from a local cache and queues writes until a connection returns. Conflicts are resolved **last-write-wins** — deliberately, because the data does not justify a merge engine and building one would consume days that hardening needs.

Airplane mode is part of the daily test routine for the AAC module, the emotion activity and routines — not an edge case discovered on demonstration day.

12 Non-functional requirements

Area	Requirement
Performance	Screen transitions under 300 ms; speech begins under 500 ms after a tap
Latency	On-device inference preferred; no per-frame network calls; assets preloaded
Offline	AAC, routines, downloaded stories and progress recording all work with no connection
Accessibility	Minimum 64 dp tap targets on child screens, WCAG AA contrast, audio labels throughout
Privacy	Minimum viable data collection; camera frames processed on-device and discarded; no raw audio or video stored; no third-party analytics on child data
Security	No hard-coded secrets; Firestore rules enforced server-side; authentication required for every write
Reliability	Every network or AI operation has explicit loading, success, empty, error and retry states
Error experience	Child screens show friendly non-technical messages; technical detail appears only in caregiver screens and logs
Localisation	Zero hard-coded user-facing strings; right-to-left layout verified for Urdu on every screen

13 Privacy, safety and ethics

These commitments are not decoration. They constrain the architecture — several of the technical choices in section 8 exist **because** of them — and they appear in the final report unchanged.

- AutoMate makes **no** diagnostic, clinical or therapeutic claim anywhere in the interface, the documentation or the presentation.
- Emotion features are labelled as a learning aid, never as a statement of fact about a person's feelings.
- Camera frames are processed on-device and immediately discarded — never uploaded, never written to storage.
- Only data strictly necessary for a feature is collected at all.
- There is no automatic behavioural labelling. Caregiver observations are free text, written by a human.
- No advertising, and no third-party analytics on child data.
- A visible privacy statement is shown during onboarding, in both languages.
- Conversation practice is scripted and branching. Open-ended generative chat for a child is out of scope on safety grounds, and a guardrail that cannot be verified will not be shipped.

The privacy story is only credible because it is **architectural**. There is no server in the speech path, no upload in the camera path, and no field in the schema for a behavioural label. Saying it on stage while the phone is in airplane mode is the proof.

14 The team

Figure 14 — Four tracks, and who blocks whom

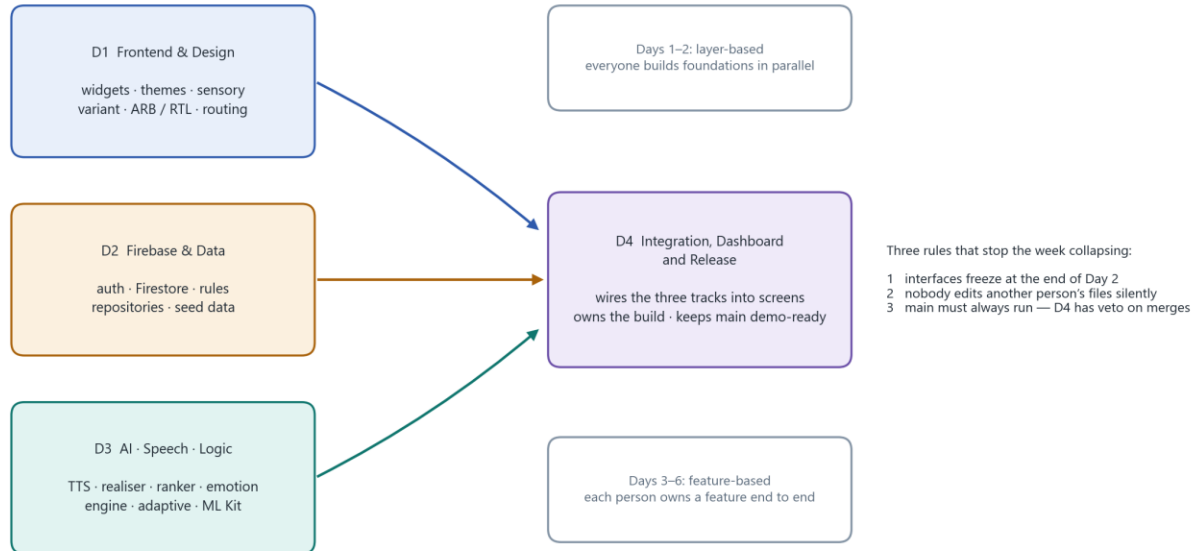


Figure 14 — Four tracks. Days one and two are layer-based so nobody is blocked; days three onward are feature-based so nobody collides.

Track	Owns	Delivers by end of day two
D1 Frontend & Design	Scaffolding, theme including the sensory variant, reusable child widgets, routing and child-mode lock, ARB localisation and RTL	Any teammate can import a child-friendly button, a loading state and a localised string, and the app switches to Urdu RTL correctly
D2 Firebase & Data	Firebase setup, authentication, Firestore schema and security rules, repositories, offline persistence, seed data	Auth works, a child profile can be created and read, and repositories return real types
D3 AI · Speech · Logic	TTS service, sentence realiser, card ranker, emotion engine, adaptive controller, ML Kit expression practice	A working bilingual TTS service on a real device, plus the activity logic with passing unit tests
D4 Integration & Release	Dashboard, progress aggregation, routine builder, gamification, integration, the build, the demo script	The app shell runs with all tabs navigable and a clean <code>flutter analyze</code>

14.1 The three rules

- 1. Interfaces freeze at the end of day two.** Signatures are published on day one marked provisional, re-published on day two marked frozen, and are untouchable afterwards without everyone's agreement.

2. **Nobody edits another person's files silently.** Merge conflicts are the single largest killer of short sprints.
3. **main must always run.** D4 has veto power on merges, and that is not negotiable in the last 48 hours.

The reason the interface freeze matters so much: publishing the contracts on day one lets three other people write code against the AI track **before the AI track exists**. It is the highest-leverage single act in the whole week. The contracts themselves are in Appendix A.

15 Delivery plan

15.1 The six-day build

Figure 15 — The real calendar: the sprint plan mapped onto hackathon dates

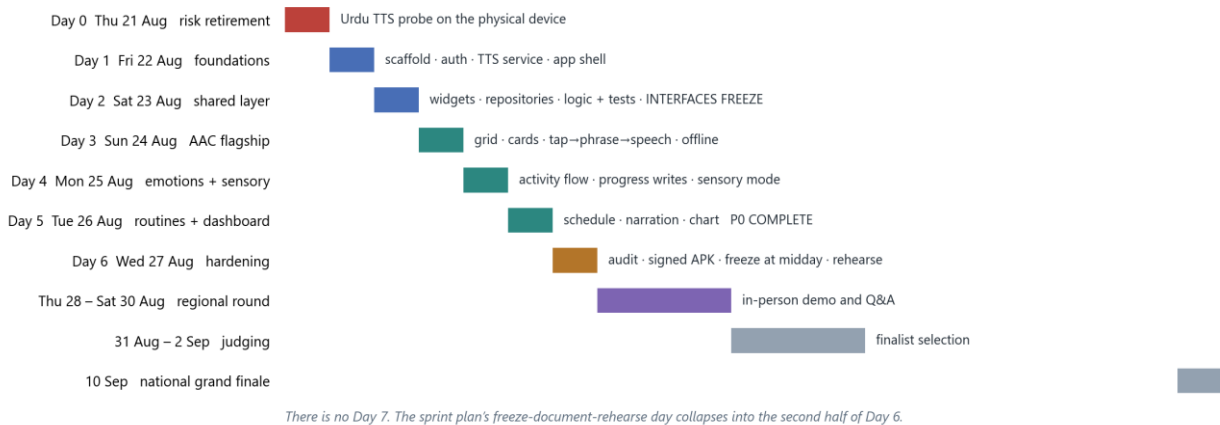


Figure 15 — The sprint mapped onto real dates. Note that the seventh day does not exist.

Day	Focus	The gate that must be true before moving on
Day 0 Thu 21 Aug	Environment and risk retirement	An Urdu voice is verified — or ruled out — on the physical demo device (section 10)
Day 1 Fri 22 Aug	Foundations, everyone in parallel	The app launches, a user can register, and a string speaks aloud in Urdu
Day 2 Sat 23 Aug	Shared layer and contracts	Nobody is blocked on anybody. Interfaces freeze. ~30 unit tests green
Day 3 Sun 24 Aug	AAC — all hands on the flagship	A child taps an apple card and the phone says “I want an apple.” In airplane mode. In Urdu
Day 4 Mon 25 Aug	Emotion activity and sensory mode	Five questions completed, stars shown, the result lands in Firestore
Day 5 Tue 26 Aug	Routines and dashboard	P0 feature-complete. Everything after this is polish and hardening
Day 6 Wed 27 Aug	Hardening, freeze at midday, rehearse	Signed APK on the demo device; two full run-throughs with no crash

Day 3 is the one that matters most. The AAC moment is the opening scene of the demonstration and it must be flawless before anything else gets attention.

15.2 The 16-week roadmap this is a slice of

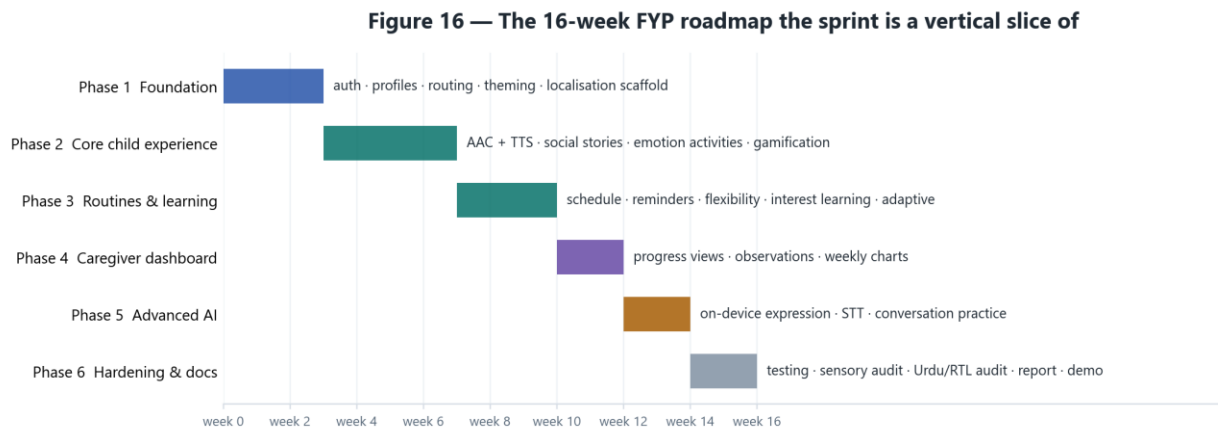


Figure 16 — The full FYP plan. The sprint builds a vertical slice through phases 1 to 4.

The sprint is not the project. The scope document describes eight modules over roughly sixteen weeks; the sprint takes a **vertical slice** through it — fewer features, each working end to end, offline, in both languages, with real persistence. That demonstrates far better than seven half-built tabs, and it is far easier to defend in a viva.

Say this to a supervisor or a judge up front: *“We scoped a vertical slice for the build week; the remaining modules are designed and documented but not implemented.”* That is a professional answer. *“We ran out of time”* is not.

15.3 If time runs short — the cut order

Decided in advance, in writing, so that a tired team at 2 a.m. does not have to decide it badly:

1. Expression practice (the camera feature) — cut first, without hesitation.
2. Routine reminders and notifications.
3. Custom card creation — ship with seeded cards only.
4. Urdu on the **parent** screens — keep Urdu on every **child** screen, because that is the accessibility claim.
5. The dashboard chart — replace it with a simple statistics list.

Never cut: the AAC communication aid with text-to-speech, or offline capability. Those two are the project’s identity. Everything else is negotiable.

16 Risks

Figure 17 — Risk heat map, with the top-right risk retired on Day 0

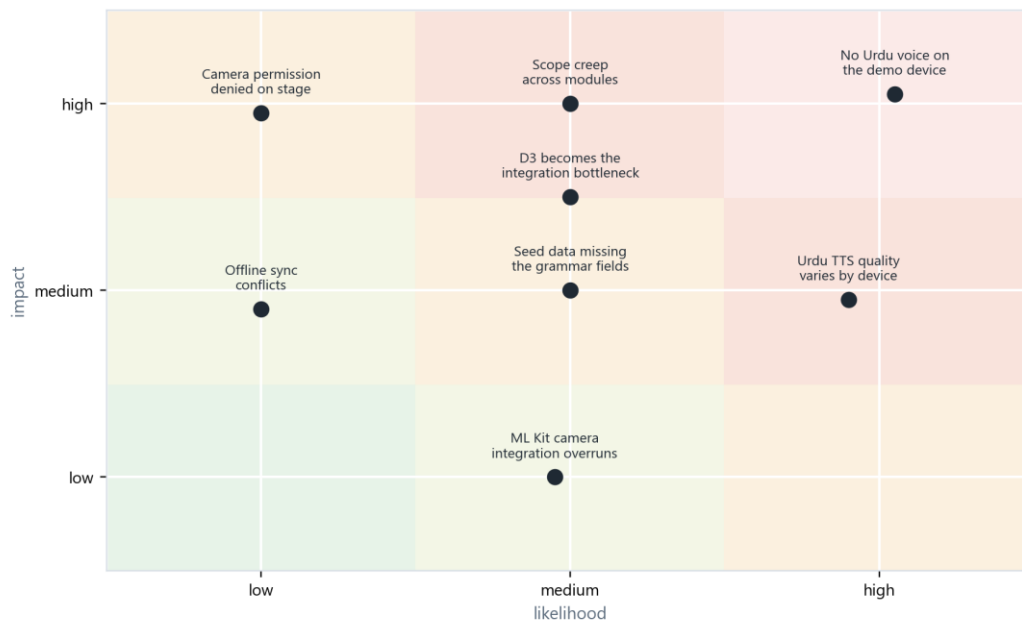


Figure 17 — The risk map. The one in the top-right corner is retired on Day 0 by the procedure in section 10.

Risk	Likelihood	Impact	Mitigation	Trigger to act
No Urdu voice on the demo device	High	High	Recorded native audio for ~30 core phrases, behind the same interface	The Day 0 probe result
Speech first-utterance latency breaches 500 ms	Medium	Medium	Warm start during <code>initialise()</code>	Day 1 measurement
Overlapping speech on rapid taps	High	Medium	Serial queue and barge-in, built in from day one	Built in, not deferred
Seed data lacks the four grammar fields	Medium	High	Requested in writing on Day 1 with the exact field list	Day 1 morning
Urdu realiser wrong on an unseen card	Medium	Medium	Test against the entire seeded card set, not just demo phrases	Day 6 audit
Scope creep across eight modules	High	High	Phase gates with explicit exit criteria; the out-of-scope list is binding	Any mid-sprint addition
ML Kit camera integration overruns	Medium	Low	Hard cut at midday on Day 6; nothing in P0 depends on it	Day 6 noon
Camera permission denied during the demo	Low	High	Granted before the demo; denial handled with a friendly fallback screen	Day 6 audit
The AI track becomes the integration bottleneck	Medium	Medium	Interfaces published Day 1, frozen Day 2, so others code against them early	Ongoing
No access to autistic children for testing	High	Medium	Validate against published accessibility guidance; seek a special-education teacher's feedback	Ongoing

17 Quality — testing and the definition of done

17.1 What is tested, and how

Type	Focus	Roughly
Unit	Sentence realiser, emotion engine, adaptive controller, card ranker, progress aggregation — all pure Dart	30+ tests
Widget	Reusable child-facing components, AAC grid, routine card, emotion activity screen	Key components
Integration	Auth → profile creation; complete-an-activity → progress-recorded	Two flows
Manual, daily	Airplane mode, Urdu and RTL layout, sensory mode audit, rapid-tap stress, first-utterance latency	Every day, on hardware

The logic track carries the team's test-coverage claim because it is the testable one: none of it needs a device, a network or Firebase. That is also what makes it safe to refactor on day five while three other people are calling into it.

17.2 Definition of done, per feature

1. Implemented within the existing feature-based architecture.
2. All user-facing strings localised in English and Urdu, with RTL verified.
3. Loading, success, empty, error and retry states all handled.
4. Child-facing errors are friendly and non-technical.
5. Respects sensory-friendly mode — including volume ceiling and reduced motion.
6. Unit and/or widget tests written and passing.
7. `flutter analyze` clean.
8. Works offline wherever the offline requirement applies.
9. Firestore rules updated if new data is touched.
10. Verified on the physical demo device, not the emulator.
11. Reviewed by at least one other team member.

18 What you hand over, and in what format

The short answer: you do not deliver a model. There is no `.tflite` file, no `.h5`, no weights, no dataset and no training notebook — because nothing in AutoMate is trained. You deliver **source code, tests, a signed APK, documentation and the report**. The only machine-learning model in the project is Google's face detector, which arrives bundled inside the ML Kit dependency: you consume it, you do not produce it.

Figure 18 — What actually gets handed over (and what does not)

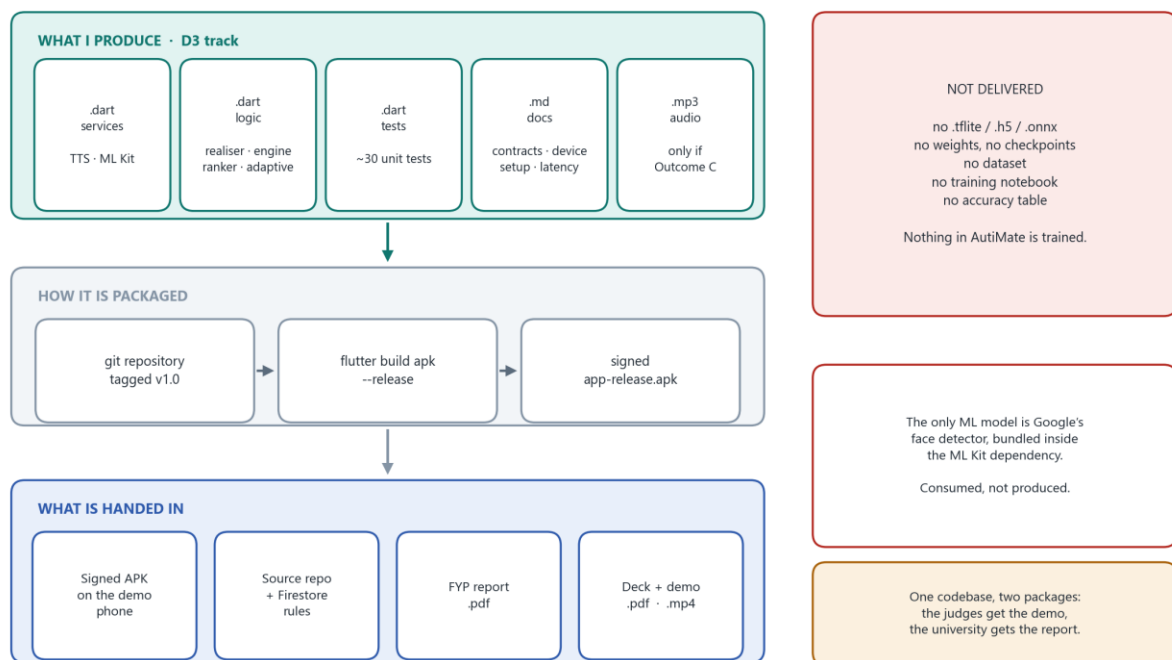


Figure 18 — Everything the AI track produces is source, tests, documents or audio. The model-shaped artefacts on the right are deliberately absent.

18.1 Why this question comes up at all

The phrase “AI project” is widely assumed to mean a trained model, a labelled dataset, a notebook and an accuracy table. That is the shape of a **research** project. AutoMate is a **product** project that uses AI where it earns its place. Both are legitimate final-year projects, and they hand over different things. Producing a model you never needed would mean you built the wrong project.

	A model-building project	AutoMate
The central artefact	A trained model file and the notebook that produced it	A working application running on a phone
Evidence of quality	Accuracy, precision and recall, a confusion matrix on a held-out split	It runs offline, speaks grammatical Urdu, starts speaking in under 500 ms, and survives two full run-throughs without a crash

	A model-building project	AutiMate
Data	A labelled dataset, cleaned and split	No dataset at all — and no child data leaves the device
Reproducibility	Re-run training and obtain comparable weights	Clone the repository, run <code>flutter run</code> , obtain the same application
The main risk	The model underperforms	The feature does not ship in time
What is defended	Architecture choices and hyperparameters	Engineering choices, and why machine learning was used only where it earned its place

18.2 What I deliver — the D3 track

Nine artefacts. Every one of them is source code, a test, a document, or — in a single conditional case — an audio asset.

#	Artefact	Format	Lives in	Who consumes it
1	<code>TtsService</code> implementation	.dart	<code>lib/core/services/tts/</code>	D4, and every screen that speaks
2	<code>SentenceRealiser</code> — English and Urdu	.dart	<code>lib/features/communication/logic/</code>	The AAC screen
3	<code>CardRanker</code>	.dart	same folder	The “frequently used” row of the grid
4	<code>EmotionActivityEngine</code> , with the confusability matrix as a <code>const</code> table	.dart	<code>lib/features/emotion_recognition/logic/</code>	The activity screen; D4’s session summary
5	<code>AdaptiveLevelController</code>	.dart	same folder	Between sessions; the parent dashboard
6	<code>ExpressionPracticeService</code> — P1 only	.dart	<code>lib/core/services/ml/</code>	The expression practice screen
7	Unit test suite, roughly 30 tests	.dart, run by <code>flutter test</code>	<code>test/</code>	The team, and the report’s quality claim
8	Frozen interface contracts, the Day 0 device note, and the measured latency numbers	.md	<code>docs/</code>	D1, D2, D4, the supervisor, the judges
9	Recorded Urdu audio — only under Outcome C	.mp3 or .m4a, mono, 44.1 kHz	<code>assets/audio/ur/</code> , declared in <code>pubspec.yaml</code>	The fallback path inside <code>TtsService</code>

Notice what is **not** on that list. The closest thing to a “model” the AI track produces is the confusability matrix — a 6×6 table of hand-authored numbers in a Dart file. That is precisely why you can draw it on a whiteboard and defend every cell in it, which no trained classifier built in six days would let you do.

18.3 What the team delivers

Deliverable	Format	Owner
Source code repository, tagged at the code freeze	git repository, plus a <code>.zip</code> export as a backup	All; D4 cuts the release
Signed Android APK	<code>.apk</code> , release build, installed on the demo phone	D4
Firebase project with deployed security rules	<code>firestore.rules</code> in the repository and deployed	D2
UI/UX designs	Figma link plus a PDF export	D1
Test suite	<code>.dart</code> ; <code>flutter test</code> green, <code>flutter analyze</code> clean	D3 leads, all contribute
Technical documentation	Markdown in <code>docs/</code> , plus this document	All
FYP report	<code>.docx</code> working copy, <code>.pdf</code> for submission	All
Presentation deck	<code>.pptx</code> plus a <code>.pdf</code> export	D4
Demo script and a recorded fallback video	<code>.md</code> plus a 1080p <code>.mp4</code>	D4

18.4 Two audiences, two packages — one codebase

This is the distinction that most often goes wrong. The hackathon and the university want different artefacts from exactly the same work.

	Hackathon regional round, 28–30 Aug	University FYP submission
What they receive	A phone with the app installed, a three-minute live demo, a short deck, a repository link	The full written report, the complete source, the documentation and the test evidence
What they judge	Does it work, is it useful, is the AI real and visible	Is the engineering sound, is the process documented, can you defend every decision
Attention available	Minutes	Hours
The centrepiece	The running APK	The report
The usual failure	A demo that crashes, or a claim that cannot be shown	Undocumented decisions and untested code

Do not hand a fifty-page document to a hackathon judge, and do not hand only an APK to a supervisor. Same work, packaged twice, with a different centrepiece each time.

18.5 Formats, names, and the things that get lost

- **APK** — name it `autimate-v1.0-demo-20260827.apk`. It must be a **release** build, signed with a keystore you have backed up somewhere other than the laptop. Lose the keystore and that build can never be updated.
- **Repository** — tag the code freeze (`v1.0-hackathon`) and also export a `.zip`. A submission that depends on GitHub being reachable at the wrong moment is a risk you do not need to take.
- **Report** — write it in `.docx`, submit `.pdf`. Place the diagrams from `docs/diagrams/` at 220 dpi rather than screenshots of them.
- **Deck** — carry a `.pdf` export alongside the `.pptx`; a venue laptop will not have your fonts.
- **Demo video** — record a clean 1080p run on Day 6 while everything still works. If the live demo fails on stage you play the video and keep talking. It is the cheapest insurance in the entire project.
- **Screenshots** — take them on Day 6, in both languages, in sensory mode, and in airplane mode with the flight icon visible in the status bar. That icon is evidence, not decoration.
- **Test evidence** — save the actual console output of `flutter test` and `flutter analyze` to text files. A claim of thirty passing tests is worth more with the output beside it.

18.6 The submission package

```
autimate-submission/
├── apk/
│   └── autimate-v1.0-demo-20260827.apk      signed release build
├── source/
│   └── autimate/                             the repository, tagged v1.0
├── docs/
│   ├── architecture.md
│   ├── setup-guide.md
│   ├── interface-contracts.md
│   ├── urdu-tts-device-notes.md
│   └── diagrams/                             frozen at the end of Day 2
│                                           the Day 0 probe result, outcome A / B / C
│                                           18 figures, 220 dpi PNG
├── report/
│   ├── AutiMate_FYP_Report.pdf              the submitted copy
│   └── AutiMate_FYP_Report.docx             the working copy
├── presentation/
│   ├── AutiMate_Deck.pdf
│   └── AutiMate_Demo.mp4                    the fallback recording
├── evidence/
│   ├── screenshots/                         EN, UR, sensory mode, airplane mode
│   ├── flutter-test.txt
│   └── flutter-analyze.txt
└── README.md                               what everything is, and how to build it
```

18.7 If someone asks “where is your model?”

“There is one machine-learning model in the app: Google’s on-device face detector, which we run locally for the expression activity — and here it is, working in airplane mode. We deliberately did not train a classifier. There is no ethically obtainable dataset of autistic children’s faces or activity performance, and a parent

needs to understand why the app did what it did. So the personalisation, the difficulty adaptation and the bilingual sentence generation are algorithms we can explain line by line, and the one place machine learning genuinely earned its place is the one place we used it. A fine-tuned emotion classifier is documented as future work."

19 The demonstration

Three minutes, rehearsed at least three times on the actual demo device, in airplane mode from the first second so that nobody has to be told it works offline — they can see it.

Time	What happens	The line that goes with it
0:00	Airplane mode is switched on, visibly, before anything else	"Everything you are about to see runs on the phone. Nothing leaves it."
0:15	A child profile is opened; the AAC grid appears	"This is what a non-verbal child sees. Eight categories, very large targets, almost no text."
0:30	Two cards are tapped: [I want] [apple] → the phone speaks	"Two taps, one sentence, spoken aloud."
0:50	The language toggle is switched to Urdu; the same two cards are tapped	"Same taps. The Urdu is generated, not translated — and the verb agrees with the child's gender."
1:15	A card is tapped repeatedly, then the grid is reloaded and it has moved to the front	"The grid learns the child's vocabulary, with a seven-day half-life so last month's habits don't crowd out today's needs."
1:35	A five-question emotion session is played, deliberately getting two wrong	"Wrong answers get visually harder as the level rises — from a matrix I can show you."
2:05	The parent dashboard shows the session that just happened	"That result was written locally and will sync when a connection returns."
2:20	Sensory mode is toggled on, on camera	"Reduced motion, muted palette, a volume ceiling. No flashing anywhere in the app."
2:35	Expression practice, if it shipped: "make a happy face", meter fills	"On-device machine learning. Frames are discarded after one pass — and we are still in airplane mode."
2:50	Close	"Support tool, not a diagnostic. Nothing about the child leaves the phone."

Prepare for the two questions that always come: **"Why isn't module X built?"** — because the week bought four modules working properly rather than eight half-working, and the rest are designed and documented. And **"Where is the AI?"** — answer it in the order given in section 8.1, claiming nothing that cannot be shown running.

20 Open questions to settle before the build starts

#	Question	Who answers	What it changes
1	Which phone is the demo device, and does it have an Urdu voice?	Whoever holds the device, on Day 0	Everything in section 9 and 10. Highest priority by a distance
2	Is a second device available with the same voice?	The team	Whether the demo has a single point of failure
3	Will the seeded cards carry <code>pos</code> , <code>isCountable</code> , <code>startsWithVowelSound</code> and <code>urduGender</code> ?	D2, on Day 1	Whether the Urdu realiser works properly or degrades to word-joining
4	Does the child profile carry gender?	D2	Urdu verb agreement. Without it, masculine is assumed and the limitation is noted
5	Who produces the six emotion face assets — photographs or illustrations?	D1, needed by Day 3	Illustrations avoid likeness and consent issues and suit the sensory-friendly design
6	Is <code>SessionResult</code> the right shape for the chart?	D4, confirm Day 1	It crosses three tracks; changing it on Day 4 is expensive
7	Does everyone have a physical Android device?	The team, before Day 0	Emulators cannot verify speech or the camera
8	Are native Urdu recordings being made regardless of the probe result?	The team	Cheap insurance against a phone that updates itself overnight

21 Glossary

Term	Meaning
AAC	Augmentative and Alternative Communication — tools that support or replace speech, here a picture-card grid that speaks
ARB	Application Resource Bundle — the JSON-like files holding translated strings in a Flutter project
Barge-in	Interrupting speech that is already playing so a new utterance can start immediately
Confusability matrix	A table of how easily each pair of emotions is mistaken for the other; used to make wrong answers harder as level rises
EMA	Exponential moving average — smoothing applied to a jittery per-frame signal before it reaches the interface
ML Kit	Google's on-device machine learning library; here, face detection with classification enabled
P0 / P1	Priority zero — non-negotiable for this build. Priority one — only if P0 finishes on schedule
Realiser	The component that turns a structure into a grammatical sentence in one specific language
RTL	Right-to-left — the text and layout direction Urdu requires
Sensory mode	A global setting that reduces motion, mutes colour, caps volume and strips visual clutter
SVO / SOV	Subject–verb–object (English) and subject–object–verb (Urdu) word order
Surface realisation	The final stage of natural language generation: producing the actual words and their order
TTS / STT	Text-to-speech (the phone speaking) and speech-to-text (the phone listening). AutoMate ships the former
Vertical slice	A small set of features built end to end rather than many features built partially

Appendix A Interface contracts

These are the signatures the other three tracks write code against. They are published on day one marked provisional and frozen at the end of day two. Everything here is pure Dart: no widget, no Firestore document, no user-facing English string.

A.1 Speech

```
enum AppLanguage { en, ur }
enum TtsState { idle, speaking, unavailable }

/// The answer to "can this device speak this language?"
class VoiceAvailability {
  final AppLanguage language;
  final bool available;
  final String? resolvedLocale; // 'ur-PK', 'ur-IN', or null
  final bool usingRecordedFallback;
}

abstract class TtsService {
  /// Call once during app start. Resolves voices and warms the engine so the
  /// first real utterance is not slowed by engine initialisation.
  Future<void> initialise();

  /// Per-language availability – for the settings screen and for the fallback.
  Future<List<VoiceAvailability>> availability();

  /// Speak now, cancelling anything already speaking (barge-in).
  /// Never throws – failures surface through [state].
  Future<void> speak(String text, AppLanguage language);

  /// Queue behind whatever is speaking. Used for routine step sequences.
  Future<void> enqueue(String text, AppLanguage language);

  Future<void> stop();

  /// D1 binds the speaker icon to this.
  Stream<TtsState> get state;
}
```

- `speak()` never throws. With no voice it emits `TtsState.unavailable` and returns; the interface shows a muted speaker and nothing crashes.
- `initialise()` must be awaited in `main()` before the first frame, or the first tap feels slow.
- The fallback chain is owned entirely inside the service. Callers never learn whether audio came from the platform engine or a recorded asset.

A.2 Sentence assembly

```
enum PartOfSpeech { carrier, pronoun, verb, noun, adjective, quantifier }
enum UrduGender { masculine, feminine }

/// The grammar-relevant subset of a card. D2 owns the full Firestore model
/// and maps into this; the logic never sees a Firestore document.
class CardGrammar {
  final String id;
  final String labelEn;
  final String labelUr;
}
```

```

    final PartOfSpeech pos;
    final bool isCountable;           // drives a / an / some in English
    final bool startsWithVowelSound; // 'apple' true, 'university' false
    final UrduGender urduGender;      // drives Urdu agreement
}

/// Grammatical facts about the child, from their profile.
class SpeakerProfile {
    final UrduGender gender;          // چاہتی vs چاہتا
}

class RealisedSentence {
    final String text;
    final AppLanguage language;
}

abstract class SentenceRealiser {
    RealisedSentence realise(List<CardGrammar> strip, SpeakerProfile speaker);
}

```

A.3 Card ranking, emotion activity, adaptive level

```

class CardUsageEvent { final String cardId; final DateTime usedAt; }

abstract class CardRanker {
    /// Top-N card ids for the "frequently used" row, most relevant first.
    List<String> rank(List<CardUsageEvent> history, {int limit = 8});
}

enum EmotionLabel { happy, sad, angry, surprised, scared, neutral }
enum SupportLevel { beginner, intermediate, advanced }

class EmotionQuestion {
    final String assetKey;           // D1 resolves this to an image
    final EmotionLabel answer;
    final List<EmotionLabel> choices; // shuffled, includes the answer
    final bool hintVisible;
    final int index;                 // 1-based, for "question 3 of 5"
    final int total;
}

class SessionResult {
    final String childId;
    final String activityType;       // 'emotion_identification'
    final int score;
    final int total;
    final SupportLevel levelPlayed;
    final SupportLevel levelAfter;   // may differ – the rule stepped it
    final Duration duration;
    final DateTime completedAt;
    final int starsAwarded;          // 0-3
}

abstract class EmotionActivityEngine {
    EmotionQuestion start({required SupportLevel level, int questionCount = 5});
    AnswerOutcome submit(EmotionLabel picked);
    EmotionQuestion? next();          // null when the session is over
    SessionResult finish();
}

abstract class AdaptiveLevelController {
    SupportLevel evaluate({
        required SupportLevel current,

```

```

    required List<bool> recentOutcomes,    // most recent last
    required bool parentLocked,
  });
}

```

The engine returns `assetKey` values and enums — **never display text**. D1 maps each enum to a localised label. That single decision is what keeps Urdu working without the logic track ever touching an ARB file.

A.4 Expression practice

```

class ExpressionReading {
  final double smile;           // 0.0 - 1.0
  final double leftEyeOpen;
  final double rightEyeOpen;
  final double headTiltDegrees;
  final bool faceDetected;
}

abstract class ExpressionPracticeService {
  Future<bool> isSupported();

  /// Emits ~10-15 readings per second while the camera streams.
  /// Frames are processed in memory and discarded - never written, never sent.
  Stream<ExpressionReading> start();

  Future<void> stop();
}

```

Appendix B The Urdu voice probe

The complete app is in the repository at [tools/urdu_tts_probe/](#), with its own README. What follows is the part that matters — the probe itself. Add one dependency, run it on a physical phone, read the report on the phone’s screen, then repeat the whole thing in airplane mode.

B.1 The dependency

```
# pubspec.yaml
dependencies:
  flutter:
    sdk: flutter
  flutter_tts: ^4.2.0
```

B.2 The probe

```
final FlutterTts tts = FlutterTts();

Future<void> probe() async {
  // 1. Which engines exist, and which one are we actually talking to?
  print(await tts.getEngines);           // Android only
  print(await tts.getDefaultEngine);

  // 2. Every locale this engine claims to know.
  final languages = (await tts.getLanguages) as List;
  final urdu = languages.where((l) => l.toString().toLowerCase().startsWith('ur'));
  print('Urdu locales: ${urdu.isEmpty ? "NONE - this is the bad outcome" : urdu}');

  // 3. The three tags the real service will try, in order.
  for (final tag in ['ur-PK', 'ur-IN', 'ur']) {
    print('isLanguageAvailable("$tag") = ${await tts.isLanguageAvailable(tag)}');
  }

  // 4. Named voices. A voice can be listed while its data is NOT downloaded,
  //    and a name ending '-network' will not work in airplane mode.
  final voices = (await tts.getVoices) as List;
  print('voices.where((v) => v.toString().toLowerCase().contains('ur-'))');
}

Future<void> speakUrdu() async {
  await tts.stop();
  await tts.setLanguage('ur-PK');        // then try 'ur-IN', then 'ur'
  await tts.setSpeechRate(0.45);         // the Android default is too fast for a child
  await tts.setPitch(1.0);
  await tts.speak('السلام علیکم ، میں سیب چاہتا ہوں');
}
```

A printed “true” is encouraging. Hearing the sentence is proof. Run `speakUrdu()` with airplane mode on, and only then record the outcome as A, B or C per section 10.6.

Appendix C Figures and sources

C.1 Figure index

Figure	Subject	Section
1	Users, modes, and what leaves the device	3
2	The eight modules and what ships this sprint	5
3	Layered architecture and track ownership	6.1
4	Feature-based project structure	6.2
5	Firestore data model	7
6	The AAC pipeline, tap to speech	8.2
7	Bilingual sentence realisation	8.3
8	Emotion activity flow	8.5
9	Adaptive difficulty state machine	8.6
10	Expression practice pipeline	8.7
11	Urdu voice resolution ladder	9.2
12	How to check for an Urdu voice	10
13	Offline-first behaviour	11
14	Team tracks and dependencies	14
15	The six-day build calendar	15.1
16	The sixteen-week roadmap	15.2
17	Risk heat map	16
18	What is handed over, and what is not	18

Source images are in [docs/diagrams/](#) at 220 dpi and can be reused directly in the FYP report and the presentation deck.

C.2 Source documents

File	What it is	How it was used here
AutoMate_Scope_Doc.md	The 16-week FYP product scope, eight modules	Sections 2–7, 12, 13, 15.2
AutoMate_7Day_Sprint_Plan.md	The four-person vertical-slice build plan	Sections 14, 15.1, 15.3, 17
AutoMate_D3_AI_Track_Plan.md	The AI / speech / logic work order	Sections 8, 9, 10, 16, Appendix A

File	What it is	How it was used here
ABC_AIH_Training_and_Programme_Schedule.pdf	The hackathon calendar	The real dates in section 15.1
tools/urdu_tts_probe/	The runnable probe app	Section 10.3, Appendix B

— End of document —